CareLink Follow-Up

Helping clinic staff find the patients who have missed an appointment — on a clinic PC, over a link that is often not there.

The problem, as I understood it

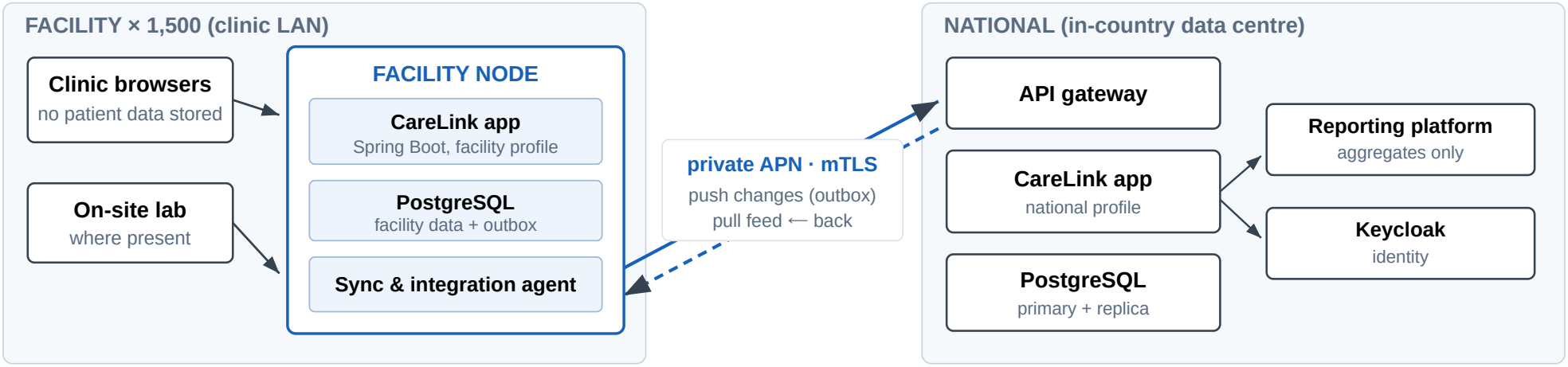
- One list, per facility, of patients whose most recent visit set an appointment they did not keep.
- It has to be right at 10,000 rows and on a six-year-old PC.
- It has to keep working while the network does not.
- It is personal health information, protected by law.

Where the brief was open, and what I assumed

- **"More than n days"** is strict: exactly 7 days is not yet listed.
- **Only the most recent visit counts.** Any later visit takes the patient off the list.
- **"Today"** is the facility's local date, not the server's.
- **Facilities have a local server.** The assumption I am least sure of — slide 4.

ARCHITECTURE

A clinic keeps working when the link does not



The path, end to end: a clinician records a visit → the row and an outbox entry are written in one local transaction, and the list updates at once → when the link is up the agent pushes the batch → national applies it *idempotently* and acknowledges → the district officer sees it. Offline, only the last step waits.

Three I would defend

1 · A server in every clinic, not an offline-first browser app

Rejected: IndexedDB in the browser. It puts patient data on shared, old PCs under tight storage quotas, and every workstation would sync 10,000 patients alone — so two PCs in one clinic drift apart from each other. **The cost I accept: hardware, and a 1,500-node fleet to operate.**

2 · The whole rule runs in SQL, never in application memory

Rejected: load the facility's patients and filter in code — which is what the extract in Section C does. One LATERAL probe per patient gives **~130 ms for any page** over 100,000 patients and 400,000 visits, measured, including the total count.

3 · Replay safety lives in the database, not in the cache

Rejected: Redis as the source of truth for idempotency. A unique constraint on the key holds across instances, restarts and a flushed cache. Redis only makes a replay cheap, fails open, and is written *after* the row is durable — so a failed insert can never suppress a real retry.

The facility node fleet

It is the right technical answer for outages that last days. It rests on two things I assumed rather than confirmed: that the Ministry can **fund** 1,500 small servers, and that districts can **support** them.

What would change my mind

- The Ministry cannot fund or support the hardware.
- Pilot data shows the 400 well-connected sites rarely actually lose their link.

What I would do instead

- Move those well-connected sites to direct online use of national.
- Route their lab traffic through national.
- Keep nodes only where connectivity genuinely demands them.

Why I am telling you this rather than defending it: the decision is reversible in one direction only. Starting with nodes and removing them later is cheap; starting online and discovering the 300 offline sites cannot work is not.

SECTION B · BUILT, NOT STUBBED

Both halves are real

100,000
seeded patients, 400,000 visits

15,566
on the largest facility's list

~130 ms
any page, measured end to end

71
automated tests, all passing

API — Java 25, reactive Spring Boot 4, PostgreSQL

- `GET /api/follow-up` to the contract; filtering, sorting, paging and counting all in SQL.
- `POST /{id}/contacted`, so "clinician writes, district officer cannot" is testable.
- Role and facility scope taken from the token, never the query string.
- Correlation ID on every request and log line; no patient data in logs — asserted by a test.

Screen — React 19, TypeScript

- Loading, empty and error states with a working retry.
- Server-side paging: **150,850** → **783** DOM elements, **2,953 KB** → **14.6 KB**.
- A real table, keyboard throughout, status as word + shape + colour.

Nothing stubbed One command — `./scripts/run.sh` — brings up the database, the API and the screen, and opens it.

What I would have blocked

	BACK-END EXTRACT	FRONT-END EXTRACT	WHY THESE
1	SQL built by string concatenation	"Mark contacted" reports success it never had	Each of these already fails in *ordinary use*, not only under attack. An apostrophe in **O'Brien** breaks the query. A clinician clicks and nothing happens. A clinic with a keyboard cannot reach the button at all. The crash-level bugs I *did not* rank first: a crash announces itself in five minutes of testing. These three ship looking fine and fail quietly in a clinic.
2	Live `sa` credentials committed in source	No error handling anywhere — "loading" for ever	
3	`catch (Exception) { /* ignore */ }` — the encounter is lost, the clinician is told it saved	`<div onClick>`, unlabelled input, status by colour alone	

I refactored the **front-end** extract: its worst problems need judgement about what a clinic needs on screen, where the back-end's have textbook answers any reviewer would give.

SECTION D · FOR THE WHOLE PANEL

Working offline, and the duplicate problem

Why duplicates happened

The clinic queued its work while the link was down. When it came back the queue was sent, timed out halfway, and was sent again *from the beginning* — so the first half arrived twice. The system had no way to recognise work it had already done.

The fix, in one sentence

Every action carries its own reference number, created once in the clinic. The server records that reference in the same breath as the work itself, so the **second arrival is recognised and ignored** — like a cheque number, not a photocopy.

The duplicates already there

Nothing is deleted. Duplicate visits are found by rule and marked superseded, staying readable.

Duplicate patients are different. Merging two patient records merges two clinical histories, and getting it wrong is a safety event. The system only *proposes* a match; someone at the clinic confirms, because they can ring the patient; the decision is recorded with a name against it, and it can be undone.

Nothing merges automatically. Ever.

NEXT

What another week would buy

1 · Row-level security in PostgreSQL

Facility scope enforced by the database as well as the application, so a future query that forgets the check still cannot cross a facility boundary. The second line of defence I designed but did not build.

2 · Keyset paging, and a measurement on real hardware

Deep pages degrade with OFFSET. And every front-end figure I have quoted is from jsdom on a developer laptop — I want the number from a six-year-old clinic PC.

3 · A test with a real screen reader

I have tested with axe and Testing Library, which catch structure, not experience. NVDA on Windows is what the clinics actually run.

4 · The sync path itself

Everything in Section D is designed and nothing of it is built. It is the largest gap between this submission and a system a Ministry could deploy.

And the honest one: I would spend part of that week deleting. The submission is longer than it needs to be, and the thinking would be easier to check if it were shorter.